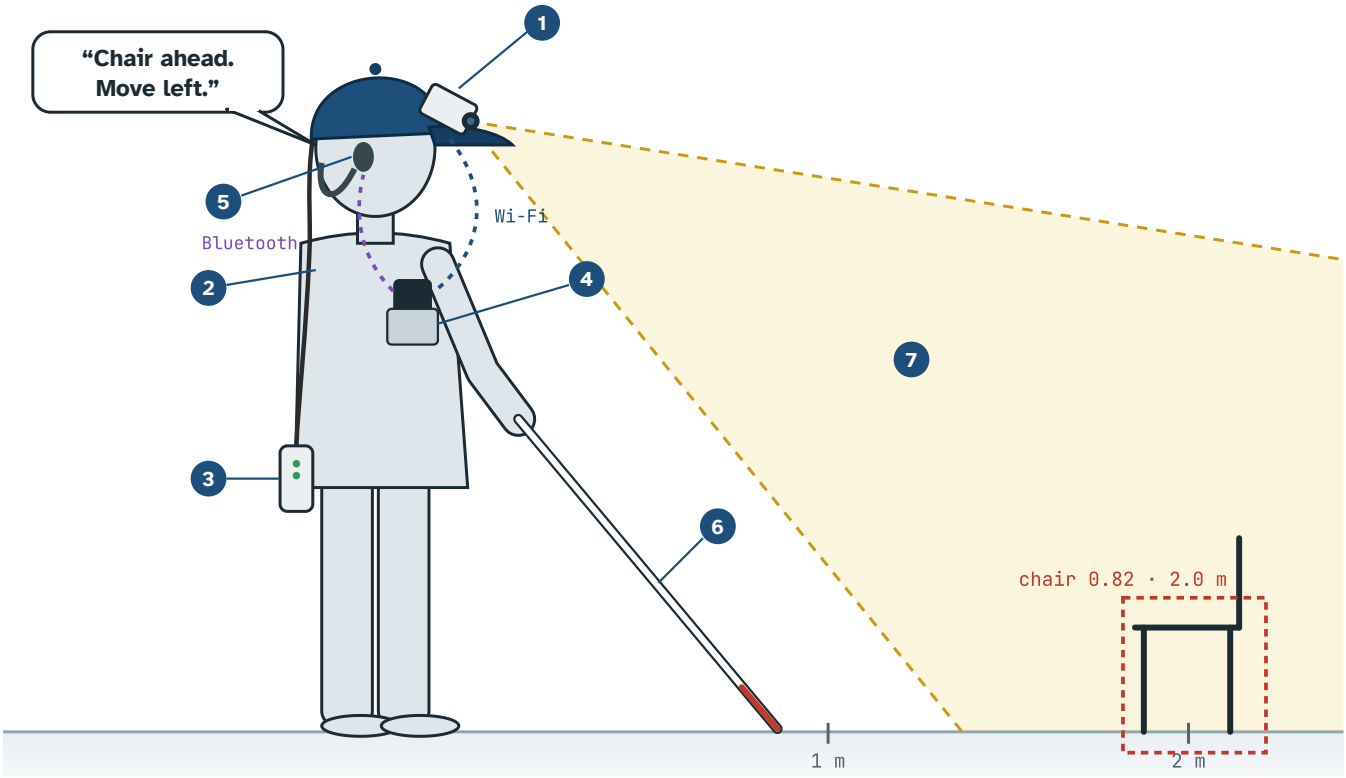
HARDWARE BUILD GUIDE · AITM BELAGAVI · DEPT. OF CSE

Camera-Based Navigation System for Visually Impaired People

Every part you need, what the finished wearable looks like, and the build steps from flashing the camera to demo day. This guide uses the AI-Thinker ESP32-CAM you already have.



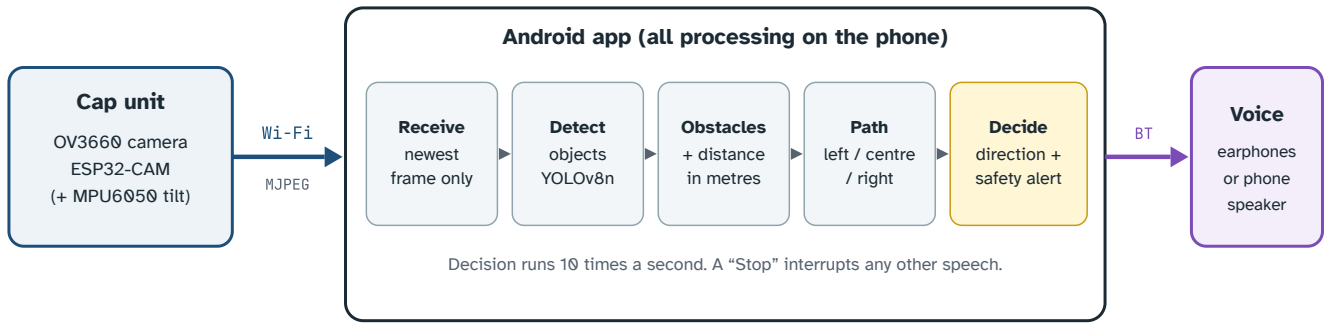
The finished system, side view. Drawn to scale for a user about 1.70 m tall (242 px = 1 m on the floor).

1. Camera unit: ESP32-CAM in a small box, tilted 25–30° down
2. Power cable running from the cap, down the back
3. 5 V power bank in a back pocket
4. Android phone running the navigation app
5. Bone-conduction earphones (ears stay open)
6. White cane, still used for the area right in front
7. Camera view: sees the floor from about 1.4 m ahead

Team	Aishwarya Kumar Telasinge · Anuja Sutar · Aishwarya Channashetti · Chandrika Bannimatti
Guide	Prof. Avanti Patil
Companion	SRS v1.0 (docs/SRS_Camera_Navigation.pdf)
Date	1 October 2026

How the finished system works

The cap camera does no thinking. It only streams video. All the AI runs on the phone, and the only output is speech.



What the user hears

SITUATION	SPOKEN INSTRUCTION
Something within 1.2 m straight ahead	“Stop. Chair ahead.”
Stairs within 4 m	“Caution. Stairs ahead.”
Something within 2.5 m, one side is clear	“Person ahead. Move left.”
All three directions blocked	“Path blocked. Stop and turn.”
Nothing nearby	“Path clear.” (at most every 6 s)
Camera switched off or out of range	“Camera disconnected. Please stop.”

Software you need on the laptop

SOFTWARE	USED FOR	COST
Arduino IDE 2.x + “esp32 by Espressif Systems” board package	Flashing the ESP32-CAM firmware	Free
Android Studio (Kotlin)	Building the navigation app	Free
Python 3.11 + Ultralytics (already set up from Phase 2)	Exporting YOLOv8n to TensorFlow Lite for the phone	Free
Project firmware `esp32_cam_ap/esp32_cam_ap.ino`	Camera hotspot and video stream (already written)	Done

> **Safety.** The system supplements the white cane; it never replaces it. Keep the power bank in a pocket, never on the cap. Stop testing if the camera box gets hot.

Every part you need

HAVE already in the project kit · BUY needed · OPTIONAL nice to have. Prices are approximate Indian online prices (Robu.in, Amazon.in) and vary by seller.



AI-Thinker ESP32-CAM (OV3660/OV2640) #1

The camera on the cap. Makes its own Wi-Fi hotspot and streams video to the phone.

HAVE —



Android phone #2

Runs the app. Android 10 or later, 4 GB RAM or more, mid-range chip. Use a team member's phone.

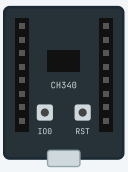
HAVE —



Laptop #3

For flashing the camera firmware and building the Android app. Your Phase-2 machine is fine.

HAVE —



ESP32-CAM-MB programmer #4

The camera clips onto it to upload firmware over micro-USB. Used on the desk only. Skip if you already have one or an FTDI adapter.

BUY ₹150–250



Slim power bank, 5000 mAh, 5 V / 2 A #5

Powers the camera from a pocket. Keeps the battery off the user's head. Runs the camera for 8+ hours.

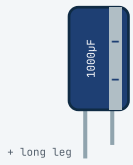
BUY ₹600–1,000



USB-A to 2-pin DuPont power cable, 1–1.2 m #6

Carries 5 V from the power bank up to the camera's 5V and GND pins. Choose a thick cable.

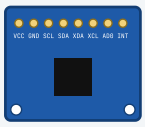
BUY ₹80–200



1000 µF 10–16 V capacitor #7

Soldered across 5V and GND at the camera. Stops the ESP32-CAM rebooting when Wi-Fi draws a burst of current.

BUY ₹10–30



MPU6050 tilt sensor (GY-521) #8

Measures how far the head is tilted, so the app can work out distance along the floor more accurately.

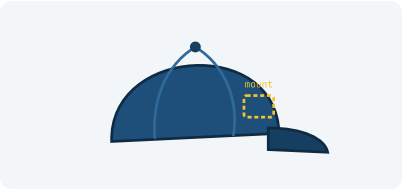
BUY ₹120–250



Female-to-female jumper wires, 10 cm (set) #9

Connects the MPU6050 to the camera board: red, black, yellow, green.

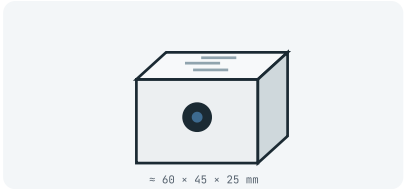
BUY ₹50–100



Baseball cap with a stiff front panel #10

The stiff front keeps the camera angle steady while walking. Avoid soft or unstructured caps.

BUY ₹200–400



Small ABS box or 3D-printed case #11

Holds the camera board and tilt sensor. Needs a lens hole at the front and air slots on top.

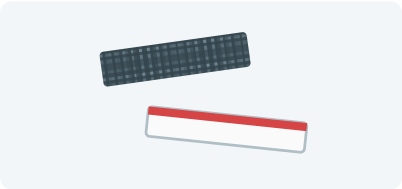
BUY ₹100–300



Bluetooth earphones, bone-conduction preferred #12

Plays the spoken instructions. Bone-conduction leaves the ears open to traffic and voices. Ordinary earbuds work for testing.

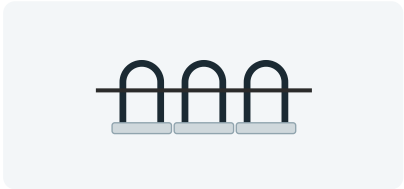
BUY ₹800–3,500



Velcro strips + double-sided foam tape #13

Velcro lets you take the camera box off the cap. Foam tape fixes the boards inside the box.

BUY ₹100–200



Small adhesive cable clips (pack) #14

Keeps the power cable flat along the cap and collar so it doesn't pull on the camera.

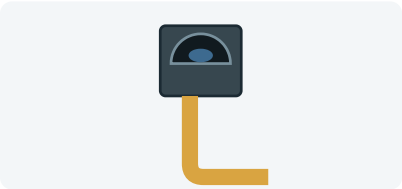
BUY ₹50–100



Stick-on aluminium heatsink, ~14 mm #15

Sits on the ESP32-S metal shield. The board gets warm when streaming and it sits on the user's head.

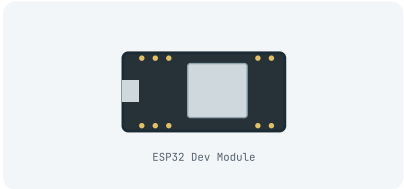
OPTIONAL ₹30–80



Wide-angle OV2640 module, 120–160° #16

Only if the standard lens misses obstacles at the sides. The app's field-of-view setting must be changed to match.

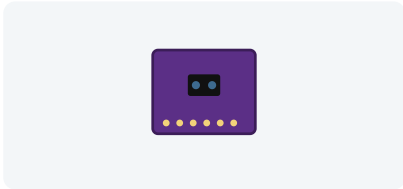
OPTIONAL ₹250–450



ESP32 Dev Module (old BLE beacon) #17

Not needed in the new design. Keep it as a spare for testing.

HAVE —

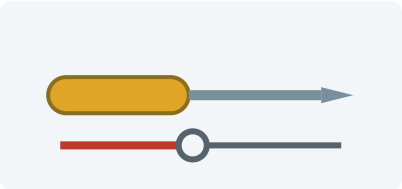


VL53L1X distance sensor (4 m) #18

A laser range reading straight ahead that catches plain walls. Adds firmware work. Only if time allows.

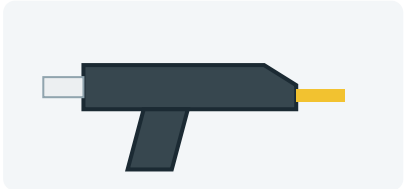
OPTIONAL ₹600–1,000

Tools (borrow from the lab)



Soldering iron, solder, small screwdriver

For the capacitor and the power-cable joint.



Hot-glue gun + craft knife or hand drill

For the lens hole, air slots and fixing the wedge.



Multimeter

To check 5 V reaches the board and the polarity is right.

Shopping list and budget

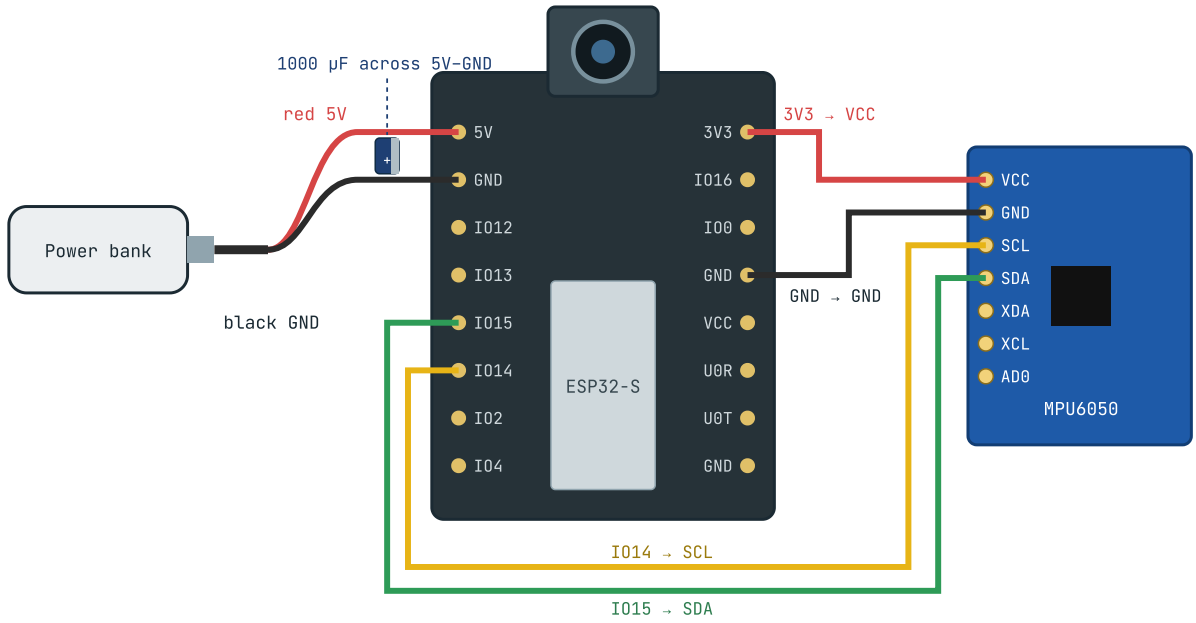
#	ITEM	QTY	APPROX. ₹	STATUS
1	AI-Thinker ESP32-CAM	1	—	HAVE
2	Android phone	1	—	HAVE
3	Laptop	1	—	HAVE
4	ESP32-CAM-MB programmer	1	150–250	BUY
5	Slim power bank 5000 mAh	1	600–1,000	BUY
6	USB-A to 2-pin DuPont power cable	1	80–200	BUY
7	1000 µF capacitor	2	10–30	BUY
8	MPU6050 (GY-521)	1	120–250	BUY
9	Female-female jumper wires	1 set	50–100	BUY
10	Baseball cap, stiff front	1	200–400	BUY
11	Small ABS box or 3D-printed case	1	100–300	BUY
12	Bluetooth earphones	1	800–3,500	BUY
13	Velcro + foam tape	1	100–200	BUY
14	Adhesive cable clips	1 pack	50–100	BUY
15	Stick-on heatsink	1	30–80	OPTIONAL
16	Wide-angle camera module	1	250–450	OPTIONAL
17	ESP32 Dev Module	1	—	HAVE
18	VL53L1X distance sensor	1	600–1,000	OPTIONAL
	Required items (#4–#14)		**2,260–6,330**	
	With all optional items		**3,140–7,860**	

Most of the range comes from the earphones. With ordinary ₹800 Bluetooth earbuds the required items cost about ₹2,300–3,600.

> **Alternative camera board.** If the ESP32-CAM keeps rebooting or the cap feels too heavy, swap it for a **Seeed XIAO ESP32S3 Sense** (₹1,400–1,800). It is thumb-sized, has USB-C, and can run from a small LiPo battery. Then you can skip items #4, #6 and #7. Only the camera pin map in the firmware changes.

Wiring diagram

Front view of the ESP32-CAM, camera at the top. The camera board is **not** on the MB programmer here; the MB is only used on the desk for flashing.



FROM	TO	WIRE	PURPOSE
Power cable red	ESP32-CAM **5V**	Red	Power in from the power bank
Power cable black	ESP32-CAM **GND** (left side)	Black	Ground
Capacitor + (long leg)	ESP32-CAM **5V**	Solder	Smooths current bursts
Capacitor – (striped side)	ESP32-CAM **GND**	Solder	Smooths current bursts
ESP32-CAM **3V3**	MPU6050 **VCC**	Red jumper	Powers the tilt sensor
ESP32-CAM **GND** (right side)	MPU6050 **GND**	Black jumper	Ground
ESP32-CAM **IO14**	MPU6050 **SCL**	Yellow jumper	I²C clock
ESP32-CAM **IO15**	MPU6050 **SDA**	Green jumper	I²C data

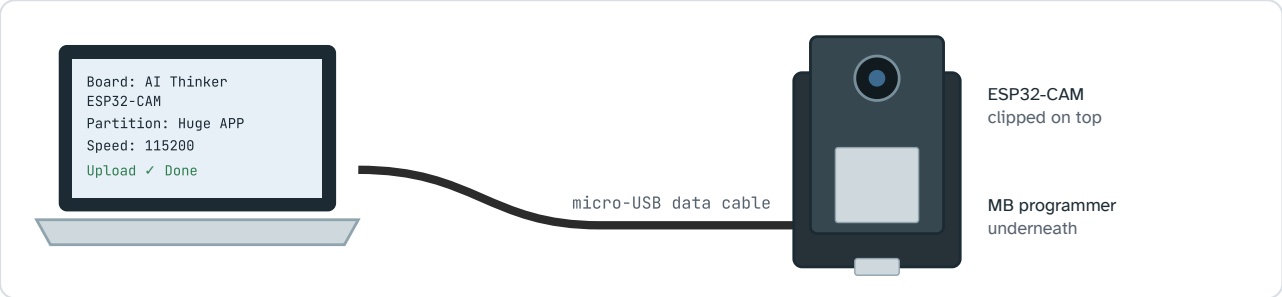
Check twice before powering on. Red to 5V and black to GND. Reversing them, or putting 5 V on the 3V3 pin, destroys the board. Measure with the multimeter first: you should read about 5.0 V between the 5V and GND pins. Do not insert an SD card; IO14 and IO15 are shared with the SD slot.

Build steps

1 Collect and check the parts

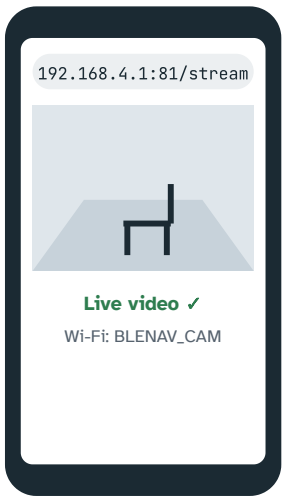
- ☐ Every **BUY** item from the shopping list has arrived.
- ☐ ESP32-CAM: the camera ribbon is fully pushed into its connector and the latch is closed.
- ☐ Power bank charged to full.
- ☐ Arduino IDE 2.x installed with the “esp32 by Espressif Systems” board package.

2 Flash the camera firmware



1. Press the ESP32-CAM onto the MB programmer, camera facing up, pins lined up. Connect it to the laptop with a micro-USB **data** cable (charge-only cables don’t work).
2. Open `esp32_cam_ap/esp32_cam_ap.ino` in Arduino IDE. Choose **Board:** AI Thinker ESP32-CAM, **Partition:** Huge APP (3MB No OTA), **Upload speed:** 115200, and the CH340 COM port.
3. Click **Upload**. If you see “Failed to connect”, hold the **IO0** button on the MB, tap **RST**, release IO0, and upload again.
4. After “Done uploading”, tap **RST** once. Open Serial Monitor at 115200; it should print that the hotspot `BLENAV_CAM` started.

3 Test the video on your phone



1. Keep the board on the MB, powered by USB.
2. On the phone, join Wi-Fi **BLENAV_CAM**, password **blenav1234**.
3. Open `http://192.168.4.1` in Chrome, then `http://192.168.4.1:81/stream`. You should see live video.
4. Wave a hand in front of the lens: the delay should be well under a second.
5. If the picture is upside down or very pink, note it; the OV3660 fix in the firmware handles this, so report it if it still happens.

4 Make the power connection

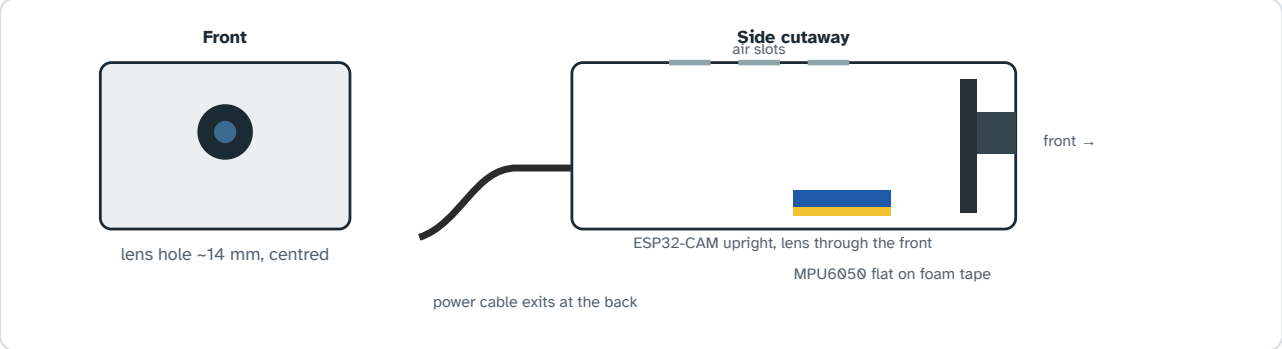
1. Remove the ESP32-CAM from the MB programmer. From now on the MB stays on the desk.
2. Solder the **1000 µF capacitor** to the top-side solder joints of the **5V** and **GND** pins (left column, first two pins). Long leg (+) to 5V. Striped side (–) to GND. Keep the legs short.
3. Push the power cable's DuPont ends onto the pins: **red → 5V**, **black → GND**.
4. Plug the USB end into the power bank. Check with the multimeter: about 5.0 V between 5V and GND.
5. Leave it streaming to the phone for **30 minutes**. Pass if the video never drops and the metal shield is warm but not too hot to touch.

> If the stream keeps dropping, the board is browning out. Use a shorter or thicker cable, check the capacitor polarity, and try another power bank. Some power banks switch off when the load is very small; the camera draws enough (about 0.2–0.3 A) that this is rare.

5 Connect the tilt sensor (MPU6050)

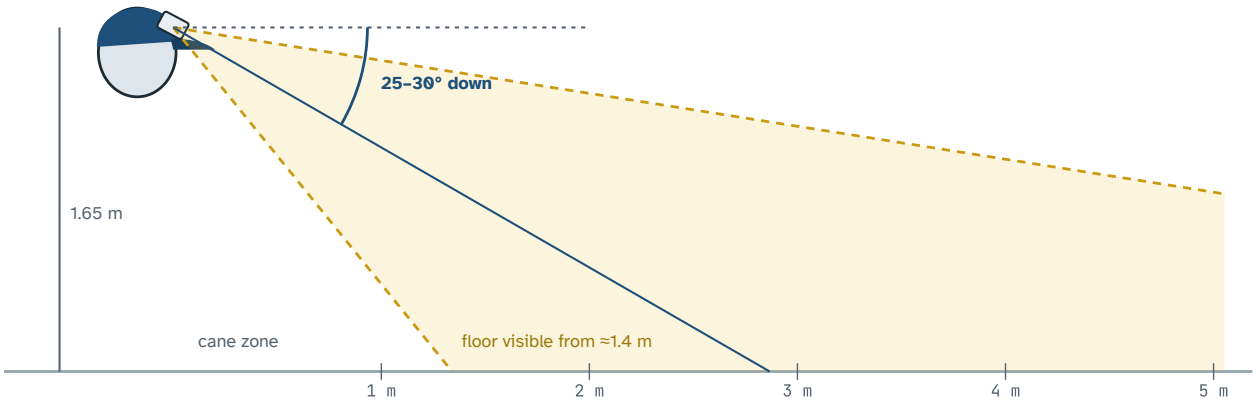
1. Solder the header pins onto the MPU6050 if they came loose.
2. Connect four jumper wires exactly as in the wiring diagram: **3V3 → VCC**, **GND → GND**, **IO14 → SCL**, **IO15 → SDA**.
3. The tilt-reading code is a firmware addition still to be written (SRS requirement FR-CAM-05). Until then the sensor can stay connected and unused; the camera works without it.

6 Build the camera box



1. Cut a ~14 mm lens hole in the front of the box. Cut 3–4 air slots in the top. Drill a small hole at the back for the cable.
2. Fix the ESP32-CAM upright behind the lens hole with foam tape so the lens looks straight out. Add the heatsink on the metal shield if you bought one.
3. Stick the MPU6050 **flat** on the bottom of the box with foam tape. It must not move relative to the camera, or the tilt reading will be wrong.
4. Secure the cable inside with a dab of hot glue so a tug on the cable can't pull the pins off.

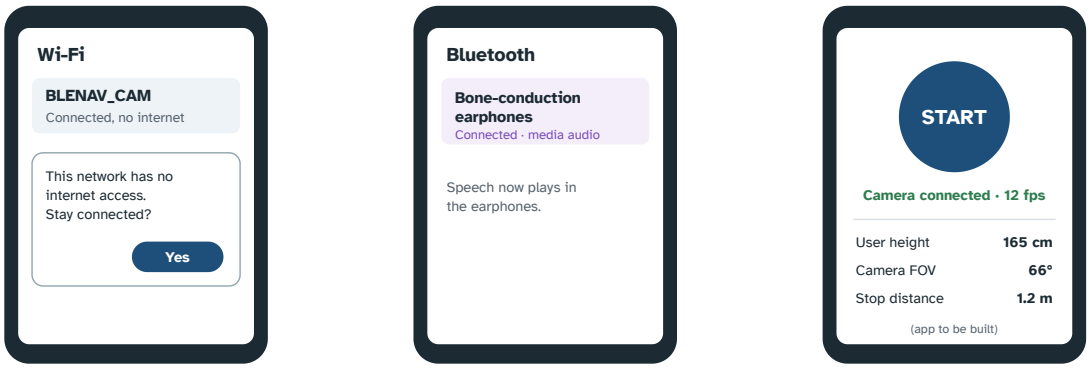
7 Mount the box on the cap



With a standard ~66° lens tilted 30° down, the camera sees the floor from about 1.4 m ahead out to the far end of a corridor. The area closer than that at floor level is the white cane's job.

1. Stick one Velcro half on the box and the other on the front panel of the cap, just above the peak, centred.
2. Glue a small foam wedge behind the box so the lens points **25–30° downward** when the user's head is level.
3. Run the cable over the top of the cap to the back strap, then down the collar to the back pocket. Fix it every 10–15 cm with cable clips. Leave a little slack at the neck so turning the head doesn't pull.
4. Put the power bank in the back pocket. Check: total weight on the cap under 100 g.

8 Set up the phone



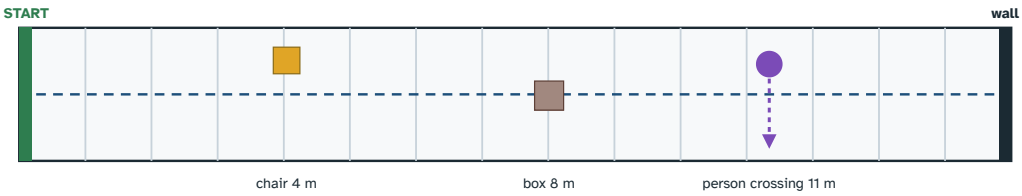
1. Pair the Bluetooth earphones (Settings → Bluetooth). Play any audio to confirm sound comes through them.
2. Join **BLENAV_CAM**. When Android says the network has no internet, tap **Yes / Stay connected**. Turn off "Switch to mobile data" / "Adaptive Wi-Fi" so the phone doesn't drop the camera.
3. Install the navigation app and enter the user's height and the camera field of view (66° for the standard lens).

9 Build the Android app in this order

Each stage is testable on its own. Use the phone's own camera or a recorded video as the source until the cap unit is ready.

STAGE	BUILD	DONE WHEN	SRS
9.1	Frame receiver: connect to BLENAV_CAM, show the stream, keep only the newest frame	Video plays in the app; unplugging the cap triggers "Camera disconnected" within 2 s	M2
9.2	Object detection: export YOLOv8n to TFLite, run it on each frame	Boxes and labels drawn on screen at 8+ fps	M3
9.3	Voice: Android TextToSpeech with priority and cooldown	"Stop" interrupts other speech; plays through the earphones	M8
9.4	Obstacles + distance: walking corridor, size-based distance (port of `egocentric.py`)	A chair at 2 m reads 1.6–2.4 m	M4, M5
9.5	Path analysis: left / centre / right zones	Chair on the left → "Move right" every time	M6
9.6	Navigation decision: port `guidance.decide()` with its tests, 10 Hz loop	Unit tests pass; full pipeline speaks correct instructions	M7
9.7	Settings screen, debug overlay, foreground service	Works with the screen off; helper can change all thresholds	M9

10 Test on a marked course



15 m corridor, tape mark every 1 m, walking line in blue

- ☐ AT-01: walking at the chair → "Chair ahead. Move ..." at about 2.5 m, "Stop" at about 1.2 m.
- ☐ AT-02: chair on the left → "Move right".
- ☐ AT-03: person crosses → warning before 1.2 m, never "Path clear" while they are in the way.
- ☐ AT-06: switch off the power bank mid-walk → "Camera disconnected. Please stop." within 2 s.
- ☐ AT-09: objects at 1, 2, 3, 4 m → distance within ±20%.
- ☐ AT-10: a blindfolded teammate walks the course with the cane and the system, no collisions.
- ☐ Record every run on video; keep the best one as the demo fallback.

11 Demo-day checklist

- ☐ Power bank and phone fully charged; spare power bank in the bag.
- ☐ Cap unit tested in the actual demo room lighting.
- ☐ Earphones paired and charged.
- ☐ Phone: Wi-Fi "stay connected" set, mobile data switching off.
- ☐ Debug view open on a second screen so evaluators can see boxes and distances.
- ☐ Recorded fallback video ready.
- ☐ White cane in hand during the demo.
- ☐ SRS, Phase-2 report and this guide printed.